

TASK1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_TOKENS 50
#define MAX_LENGTH 200

int parse_single_quotes(char *input, char tokens[][MAX_LENGTH])
{
    int i = 0;
    int token_count = 0;

    while (input[i] != '\0')
    {
        /* Skip spaces */
        while (isspace((unsigned char)input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        char token[MAX_LENGTH];
        int j = 0;

        /* Handle single quoted string */
        if (input[i] == '\')
        {
            i++; // Skip opening quote

            while (input[i] != '\0' && input[i] != '\')
            {
                if (j < MAX_LENGTH - 1)
                {
                    token[j++] = input[i];
                }

                i++;
            }

            /* Check closing quote */
            if (input[i] == '\')
            {
                i++;
            }
            else
            {
                printf("\nERROR: Missing closing single quote!\n");
                return -1;
            }

            token[j] = '\0';

            /* Store quoted string */
            strcpy(tokens[token_count], token);
            token_count++;
        }
        else
        {
            /* Normal unquoted token */
            while (input[i] != '\0' &&
                !isspace((unsigned char)input[i]))
            {

```

GNU nano 7.2

```
void validate_results(char tokens[][MAX_LENGTH], int count)
{
    printf("\n===== PARSING RESULTS =====\n");

    if (count == 0)
    {
        printf("No tokens found.\n");
        return;
    }

    printf("Parsing successful!\n");
    printf("Total tokens: %d\n\n", count);

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\n", i + 1, tokens[i]);
    }

    printf("=====\n");
}

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_LENGTH];

    printf("=====\n");
    printf("      SINGLE QUOTE PARSER IN C\n");
    printf("=====\n");

    printf("\nRules:\n");
    printf("1. Single quotes preserve literal content.\n");
    printf("2. Variable expansion is ignored.\n");
    printf("3. Spaces inside quotes are preserved.\n");
    printf("4. Quoted strings are stored as one token.\n");

    printf("\nEnter a command:\n> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Input error.\n");
        return 1;
    }

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("ERROR: Empty input.\n");
        return 1;
    }

    int count = parse_single_quotes(input, tokens);

    if (count == -1)
    {
        printf("\nParsing failed.\n");
        return 1;
    }

    validate_results(tokens, count);

    printf("\nProgram completed successfully.\n");

    return 0;
}
```

```

satwika@SATWIKAG:~$ nano quote_parser.c
satwika@SATWIKAG:~$ gcc quote_parser.c -o quote_parser
satwika@SATWIKAG:~$ ./quote_parser
=====
SINGLE QUOTE PARSER IN C
=====

Rules:
1. Single quotes preserve literal content.
2. Variable expansion is ignored.
3. Spaces inside quotes are preserved.
4. Quoted strings are stored as one token.

Enter a command:
> echo 'Hello World'

===== PARSING RESULTS =====
Parsing successful!
Total tokens: 2

Token 1: [echo]
Token 2: [Hello World]
=====

Program completed successfully.
satwika@SATWIKAG:~$ |

```

TASK_2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 200

/* Expand environment variables such as $USER and $HOME */
void expand_variable(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_TOKEN_LENGTH - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++; // Skip $

            /* Read variable name */
            while ((isalnum((unsigned char)input[i]) ||
                input[i] == '_' &&
                k < 99)
            {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            /* Get environment variable */
            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int x = 0;
                    value[x] != '\0' &&
                    j < MAX_TOKEN_LENGTH - 1;
                    x++)
                {
                    output[j++] = value[x];
                }
            }
            else
            {
                /* Variable does not exist */
                output[j++] = '$';

                for (int x = 0;
                    variable[x] != '\0' &&
                    j < MAX_TOKEN_LENGTH - 1;
                    x++)
                {
                    output[j++] = variable[x];
                }
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

/* Parse command and handle double quotes */
int parse_command(char *input,

```

```

GNU nano 7.2
    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\n",
               i + 1,
               tokens[i]);
    }

    printf("\n=====\\n");
}

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];

    printf("=====\\n");
    printf("    DOUBLE QUOTE PARSER IN C\\n");
    printf("=====\\n");

    printf("\\nRules:\\n");
    printf("1. Double quotes preserve spaces.\\n");
    printf("2. Variables are expanded inside quotes.\\n");
    printf("3. Quoted text is stored as one token.\\n");
    printf("4. Missing quotes are detected.\\n");

    printf("\\nEnvironment variables:\\n");

    char *user = getenv("USER");
    char *home = getenv("HOME");

    printf("USER = %s\\n",
           user != NULL ? user : "Not set");

    printf("HOME = %s\\n",
           home != NULL ? home : "Not set");

    printf("\\nEnter a quoted command:\\n> ");

    if (fgets(input,
              sizeof(input),
              stdin) == NULL)
    {
        printf("Input error.\\n");
        return 1;
    }

    /* Remove newline */
    input[strcspn(input, "\\n")] = '\\0';

    if (strlen(input) == 0)
    {
        printf("ERROR: Empty input.\\n");
        return 1;
    }

    /* Parse command */
    int count = parse_command(input, tokens);

    if (count == -1)
    {
        printf("\\nParsing failed.\\n");
        return 1;
    }

    /* Validate result */
    validate_output(tokens, count);

    printf("\\nProgram completed successfully.\\n");

    return 0;
}

```

```
Program completed successfully.
satwika@SATWIKAG:~$ nano double_quotes.c
satwika@SATWIKAG:~$ gcc double_quotes.c -o double_quotes
satwika@SATWIKAG:~$ ./double_quotes
```

```
=====
DOUBLE QUOTE PARSER IN C
=====
```

Rules:

1. Double quotes preserve spaces.
2. Variables are expanded inside quotes.
3. Quoted text is stored as one token.
4. Missing quotes are detected.

Environment variables:

USER = satwika

HOME = /home/satwika

Enter a quoted command:

> echo "Hello World"

```
===== VALIDATION =====
```

Parsing successful!

Total tokens: 2

Token 1: [echo]

Token 2: [Hello World]

```
=====
Program completed successfully.
```

```
satwika@SATWIKAG:~$ |
```